

Backtesting Engine Software Requirements Specification

1. Introduction

1.1 Purpose

The purpose of this software is to simulate a trading algorithm on historical sets of financial data to test the algorithm's trading performance. This backtesting engine will allow a user to upload an algorithm of their choice, and test it on a variety of datasets which will be provided in the engine itself.

1.2 Scope

This backtesting engine will support the following:

- Selecting from a predefined list of built-in trading strategies (implemented as Python modules)
- Loading of provided historical financial datasets (e.g. stock data)
- Execution of the strategy against the dataset, simulating trades over time
- Tracking portfolio state, which will include cash, positions, and profits/losses
- Display of performance metrics such as total return
- Visualization tools including equity curves and trade markers

This system is intended for simple educational and research purposes, as it will be a standalone Python application

1.3 Definitions, Acronyms, and Abbreviations

- Backtesting Engine: A software tool used to evaluate the effectiveness of trading strategies using historical market data.
- Positions: The quantity of a security held in a trading account at any point in time (long, short, or neutral).
- Equity Curves: Graphs that depict the value of a trading account over time, used to visualize strategy performance.
- Trade Markers: Visual indicators on charts that denote where buy and sell actions were executed.
- SRS: Software Requirement Specification
- CSV: Comma-Separated Values
- PnL: Profit and Loss
- MA: Moving Average

- API: Application Programming Interface

1.4 Intended Audience and Reading Suggestions

This document is intended for developers and testers who are interested in quantitative finance and software development. It is structured to define functional and non-functional requirements, system features, and high-level design assumptions

1.5 References

- [1] Investopedia. (n.d.). Backtesting Definition.
<https://www.investopedia.com/terms/b/backtesting.asp>
- [2] Backtrader Documentation. (n.d.). <https://www.backtrader.com/docu/>
- [3] Zipline Documentation. (n.d.). <https://zipline.ml4trading.io/>
- [4] QuantConnect Documentation. (n.d.). Backtesting.
<https://www.quantconnect.com/docs/v2/cloud-platform/backtesting>
- [5] Mayer Krebs. (2024). Building a Backtesting Framework in Python from Scratch.
<https://mayerkrebs.com/building-a-backtesting-framework-in-python-from-scratch/>

2. Overall Description

2.1 Product Perspective

This system will be a self-contained application written in Python. It will rely on external libraries such as pandas for data handling and matplotlib for visualization.

2.2 Product Functions

- Read historical financial data
- Select from a predefined list of built-in trading strategies
- Simulate market behavior and execute trades
- Track performance and portfolio metrics
- Generate and export performance reports

2.3 User Classes and Characteristics

- Developer Users: Able to write or modify predefined algorithmic strategy modules in Python
- Analytical Users: Users who interpret strategy outcomes through performance metrics, visualizations, and trade summaries

2.4 Operating Environment

- Python 3.9+

- Operating Systems: macOS, Windows, Linux
- Dependencies: pandas, numpy, matplotlib

2.5 Design and Implementation Constraints

- Python-only implementation
- Local execution only (No real-time data fetching while testing)
- Limited to provided datasets and format capabilities (e.g. CSV)
- No GUI
- No real-time support
- Output is both visual (matplotlib .PNGs) and CSV (trade, equity, metrics)

2.6 Assumptions and Dependencies

- User has basic programming experience
- The historical data is correctly formatted and free of major errors
- No External APIs are required, execution is fully local

3. Functional Requirements

- Select Strategy
 - The system shall allow the user to select a predefined trading strategy from a list of available modules.
- Select Ticker
 - The system shall allow the user to select one or more ticker symbols from a predefined list of local CSV files.
- Run Simulation
 - The system shall simulate the selected strategy over the selected dataset, applying logic on a daily basis to generate BUY, SELL, or HOLD signals.
- Execute Trades
 - The system shall simulate execution of trades based on generated signals, updating cash balance and position holdings accordingly.
- Track and Record Portfolio State
 - The system shall record daily portfolio state including cash, equity, holdings, and executed trades.
- Visualize Results
 - The system shall generate visual outputs including an equity curve with overlaid trade markers and shaded position intervals.
- Output Performance Metrics
 - The system shall calculate total return, maximum drawdown, Sharpe ratio, and win rate, and display them in the terminal and export them to text.
- Export Results

- The system shall export trade logs, equity curves, and performance metrics to structured CSV and PNG files within a results folder.
- Generate Summary Report
 - The system shall create a summary CSV report aggregating results for all ticker-strategy combinations run in a session.

4. Non-Functional Requirements

- Portability
 - The system shall run on Windows, macOS, and Linux systems with Python 3.9+.
- Usability
 - The system shall have clear prompts and instructions for uploading scripts and selecting datasets.
- Performance
 - The system shall complete a single backtest on 1 year of daily data within 5 seconds.
- Reliability
 - The system shall validate inputs and handle invalid strategy files or missing data gracefully without crashing.
- Maintainability
 - The system shall use a modular codebase to allow future enhancements like slippage, fees, or live data integration.
- Data Privacy
 - The system shall not upload any user data to external servers.
- Scalability
 - The system shall support multiple assets and timeframes without requiring architecture changes.